University of Science and Technology of Oran Mohamed Boudiaf
Faculty of Mathematics and Informatics
Department of Computer Science

Why DevOps?

From the Dev vs Ops conflict to a culture of fast, reliable delivery

---

Mohammed Said Belaid
Eng 5 SECI, DevOps

Opening

Today's agenda

1. A story: 45 minutes, $440 million
2. How software used to ship
3. The Dev vs Ops problem
4. The birth of DevOps
5. What DevOps is (and is not)
6. Measuring DevOps
7. The DevOps lifecycle and this course

> Today's question: *why* do we need DevOps at all? The *how* comes in the rest of the course.

A story: Knight Capital, August 1st, 2012

- Knight Capital: one of the largest US stock traders (~17% of NYSE volume)
- Night before: new trading code deployed by hand to 8 servers
- One server was not updated: still running old, dead code
- A reused configuration flag reactivated that dead code
- 9:30 market opens: the 8th server sends millions of unwanted orders
- 97 automated warning e-mails before opening: nobody acted on them
- 45 minutes to stop it $\rightarrow$ $440 million lost
- The company was sold a few months later

Source: U.S. SEC, Administrative Proceeding File No. 3-15570 (2013)

What went wrong?

Was this a *developer* bug? An *operations* mistake? Both? Neither?

What went wrong?

Was this a *developer* bug? An *operations* mistake? Both? Neither?

- Manual, unrepeatable deployment
- No automated check that all servers were identical
- Dead code kept in production, flags reused
- Alerts that nobody watched or understood
- No quick, rehearsed way to roll back

What went wrong?

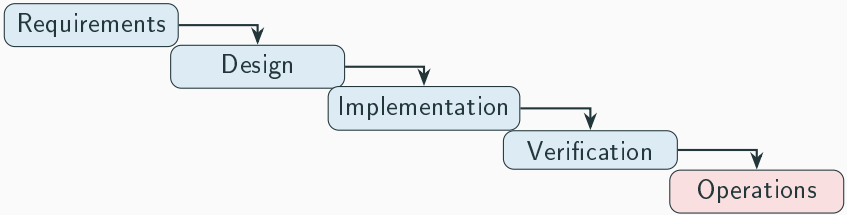
Was this a *developer* bug? An *operations* mistake? Both? Neither?

- Manual, unrepeatable deployment
- No automated check that all servers were identical
- Dead code kept in production, flags reused
- Alerts that nobody watched or understood
- No quick, rehearsed way to roll back

> None of these is a "coding" problem. They are problems of process, automation, feedback and collaboration, which is exactly what DevOps is about.

How software used to ship

The traditional lifecycle: Waterfall



- Sequential phases, each handed to the next team
- Releases every 6 to 18 months
- Feedback from real users arrives... at the very end
- Irony: W. Royce (1970), often cited as its inventor, described it as "risky and invites failure"

Agile (2001): closing the Business-Dev gap

- *Manifesto for Agile Software Development* (2001)
- Short iterations (sprints of 1-4 weeks)
- Customer collaboration, working software, responding to change
- Scrum, XP, Kanban...



Agile (2001): closing the Business-Dev gap

- *Manifesto for Agile Software Development* (2001)
- Short iterations (sprints of 1-4 weeks)
- Customer collaboration, working software, responding to change
- Scrum, XP, Kanban...



Dev now produces working software every 2 weeks... but it still reaches production every 6 months.

“Done” is not “delivered”

“It’s done, it just isn’t deployed yet.”

- A feature has zero value until users can use it
- Code waiting to be deployed = inventory (Lean): it costs money, hides bugs, and gets stale
- Agile optimised a part of the chain; the bottleneck moved to delivery and operations

> Lean idea: optimise the whole value stream, from idea to user, not each team separately.

The Dev vs Ops problem

Two teams, two missions

Dev

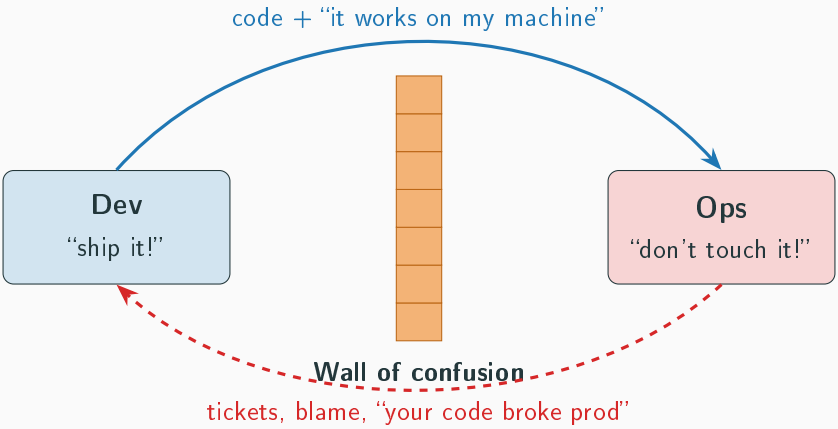
- Mission: deliver change (new features, fixes)
- Rewarded for: speed, features shipped
- Environment: laptop, test server
- Sees production as: "someone else's problem"

Ops

- Mission: keep things stable (availability, security)
- Rewarded for: uptime, no incidents
- Environment: real servers, real load, real data
- Sees change as: "the main cause of outages"

Both are rational. Their goals are structurally opposed:
every change Dev wants is a risk Ops must absorb.

The “wall of confusion”



Symptom 1: “It works on my machine”

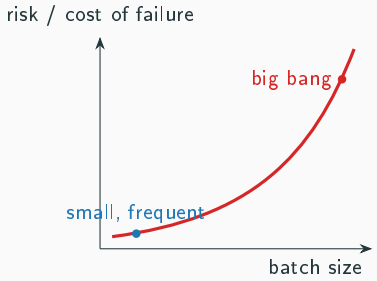
- Dev environment $\neq$ test environment $\neq$ production environment
  - different OS versions, libraries, configuration, data, network
- Installation instructions in a Word document or a wiki page (outdated)
- Bugs that appear *only* in production, and nobody can reproduce them

> Root cause: the environment is not part of what is versioned, tested and delivered.

→ Answers: containers and Infrastructure as Code

Symptom 2: rare, big, manual releases

- "Release weekend": 40-step runbook, 10 people, pizza, stress
- Months of changes deployed at once
- When it fails: which of the 500 changes broke it?
- Rollback is hard or impossible



Risk grows with batch size, not with frequency. Small changes are easier to test, understand and undo.

Symptom 3: snowflake servers

- Servers configured by hand, over years, by different people
- Each server is unique, fragile, impossible to rebuild identically
- "Don't reboot that one, nobody knows how it works"
- Configuration drift: two "identical" servers slowly become different (remember Knight Capital!)

"Pets vs cattle": we name and nurse pets; cattle are numbered and replaceable.

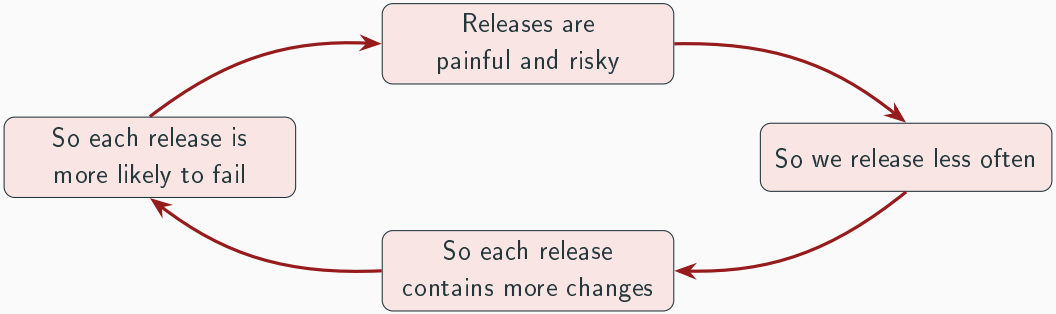
→ Answers: orchestration and Infrastructure as Code

Symptom 4: silos and blame

- Each team optimises its own metric, not the global outcome
- Communication through tickets and hand-offs: waiting time everywhere
- Incident $\rightarrow$ "who did it?" $\rightarrow$ people hide mistakes
- Hidden mistakes $\rightarrow$ no learning $\rightarrow$ the same incident again

A culture of fear makes systems less safe: problems are reported late, or never.

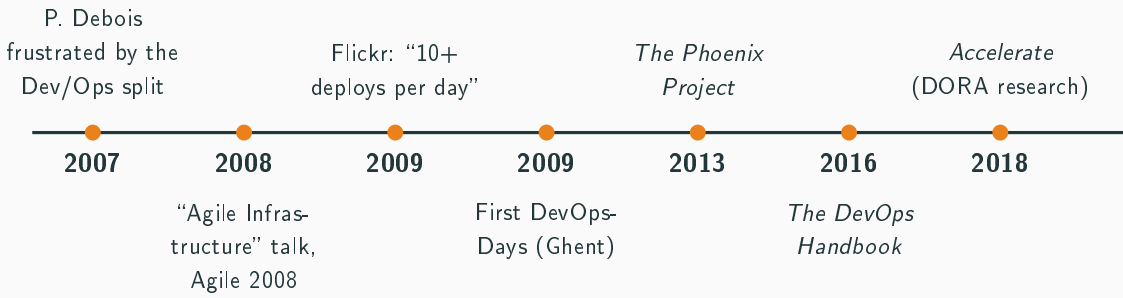
The vicious circle



DevOps reverses it: "If it hurts, do it more often" (and automate it).

The birth of DevOps

A short history



- The word "DevOps" comes from the name of the conference: *DevOpsDays*
- A movement started by practitioners, not by a vendor or a standard

Flickr, 2009: "10+ deploys per day"

J. Allspaw (Ops) and P. Hammond (Dev), O'Reilly Velocity conference

Tools

- Automated infrastructure
- Shared version control
- One-step build and deploy
- Feature flags
- Shared metrics
- Chat and bots (IRC)

Culture

- Respect
- Trust
- Healthy attitude about failure
- Avoiding blame

> Dev and Ops share one goal: "enable the business". Half of the solution is culture.

What DevOps is (and is not)

Defining DevOps

> *"DevOps is a set of practices intended to reduce the time between committing a change to a system and the change being placed into normal production, while ensuring high quality."*
>
> *Bass, Weber, Zhu, DevOps: A Software Architect's Perspective, 2015*

Working definition for this course:

> DevOps is a culture and a set of practices that bring development and operations together, so that software can be delivered quickly, safely and repeatedly, with fast feedback from production.

What DevOps is NOT

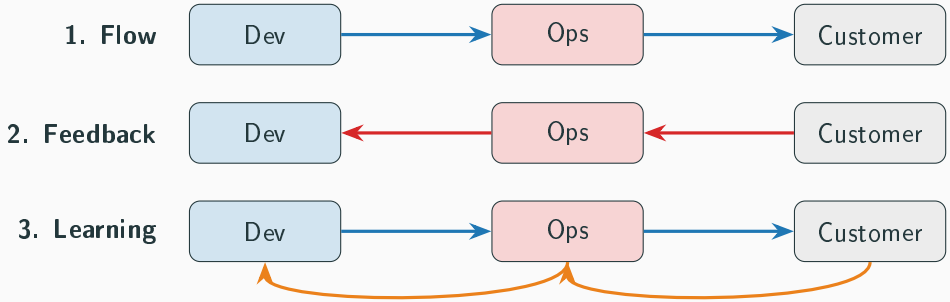
- Not a tool: installing Jenkins or Kubernetes does not make you "DevOps"
- Not a job title: a "DevOps engineer" alone cannot fix a siloed organisation
- Not a new silo: a "DevOps team" between Dev and Ops = a third wall
- Not only automation: automating a bad process gives a fast bad process
- Not "NoOps": operations do not disappear, they become shared and automated
- Not only for start-ups: banks, governments, industry do it too

CALMS: the five pillars

Pillar	Idea	Example
Culture	Shared responsibility, trust, no blame	Dev is on call for its service
Automation	Remove manual, repetitive, risky work	CI/CD pipeline, IaC
Lean	Small batches, limit work, flow	Small pull requests, daily deploys
Measurement	Decisions based on data	DORA metrics, monitoring
Sharing	Knowledge, tools and goals are shared	Shared dashboards, postmortems

CAMS proposed by J. Willis and D. Edwards (2010); "L" for Lean added by J. Humble.

The Three Ways (*The DevOps Handbook*)



1. Flow: make work move fast from left to right (small batches, automation)
2. Feedback: detect problems early, from right to left (tests, monitoring)
3. Continual learning and experimentation: take risks, learn from failure, improve

Blameless postmortems

"The question is not *who* caused the failure, but *how* our system allowed it."

- After each incident, write a postmortem: timeline, impact, causes, actions
- Assume people acted reasonably with the information they had
- Fix the system (process, tooling, alerts), not the person
- Share it widely: everybody learns

> Knight Capital again: "the technician forgot a server" is a symptom. The cause is a deployment process that *allowed* forgetting a server.

Measuring DevOps

Why measure?

- "Without data, you're just another person with an opinion" (W. E. Deming, attributed)
- Measure outcomes (value for users), not outputs (lines of code, number of commits)
- Measure teams and systems, never individuals

> Goodhart's law: "When a measure becomes a target, it ceases to be a good measure."

The four DORA metrics

Throughput (speed)	Stability
Deployment frequency How often do we deploy to production?	**Change failure rate** % of deployments causing a failure
Lead time for changes Time from commit to running in production	**Time to restore service** Time to recover from a failure

DORA = DevOps Research and Assessment (N. Forsgren, J. Humble, G. Kim), *Accelerate*, 2018; yearly *State of DevOps* reports.

Speed vs stability: a false trade-off

Intuition (Ops view):

going faster means breaking more.

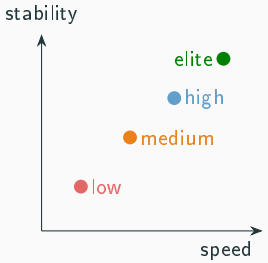
Speed vs stability: a false trade-off

Intuition (Ops view):

going faster means breaking more.

Data (State of DevOps 2019), elite vs low performers:

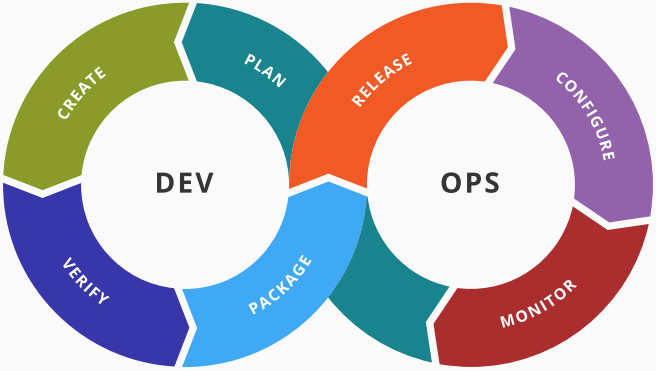
- 208× more frequent deployments
- 106× faster lead time
- 2604× faster recovery
- 7× lower change failure rate



> The best teams are both faster and more stable. The practices that create speed (small batches, automation, fast feedback) also create stability.

The DevOps lifecycle

The DevOps infinity loop



A continuous loop: no “end of project”, no hand-off. One team, one flow, constant feedback.

Image: Kharnagy, “Devops-toolchain”, Wikimedia Commons, CC BY-SA 4.0

From problems to practices

Problem (WHY)	Practice	Tool
Lost work, no history, "who changed?"	Version control, code review	Git, GitHub
Integration hell, late bugs	Continuous Integration	GitHub Actions
Risky, manual release weekends	Continuous Delivery	GitHub Actions
"Works on my machine"	Containers	Docker
Scaling, failures, many servers	Orchestration	Kubernetes
Snowflake servers, drift	Infrastructure as Code	Terraform
"We learn from users' complaints"	Monitoring, observability	Prometheus, Grafana

Course roadmap

Lecture	Lab
Why DevOps?	(no lab)
Version control: why history matters	Git internals, branches, conflicts
Collaboration and quality: reviews, tests	GitHub flow, pull requests
Continuous Integration	GitHub Actions: build and test
Continuous Delivery and Deployment	Release pipeline, artifacts
Containers	Dockerfile, images, containers
Multi-container apps, registries, 12-factor	Docker Compose, push from CI
Orchestration	Kubernetes: deployments, services
Infrastructure as Code	Terraform
Monitoring and observability; wrap-up	Prometheus and Grafana

Organisation

Practical

- 10 lectures (1h30) + 9 labs (1h30)
- Labs in pairs, same pair all semester
- One shared GitHub repository per pair
- Assessment: to be announced

Before next session

- Create a GitHub account
- Check `git --version` on your machine
- Choose your lab partner
- Read: the Flickr talk slides (2009)

Key takeaways

1. Dev and Ops had opposed goals: change vs stability → a wall of confusion
2. Big, rare, manual releases are more risky, not less
3. DevOps is a culture + practices + tools, in that order
4. CALMS and the Three Ways: flow, feedback, learning
5. Speed and stability go together (DORA)
6. Every tool in this course answers a problem: always ask "why?"

Further reading

- G. Kim, K. Behr, G. Spafford, *The Phoenix Project*, IT Revolution, 2013 (novel)
- G. Kim, J. Humble, P. Debois, J. Willis, *The DevOps Handbook*, IT Revolution, 2016 (2nd ed. 2021)
- N. Forsgren, J. Humble, G. Kim, *Accelerate*, IT Revolution, 2018
- J. Humble, D. Farley, *Continuous Delivery*, Addison-Wesley, 2010
- B. Beyer et al., *Site Reliability Engineering*, O'Reilly, 2016 (free online)
- J. Allspaw, P. Hammond, "10+ Deploys Per Day", Velocity 2009 (slides online)
- DORA, *State of DevOps Reports*, `https://dora.dev`

Next session

Version control

Why do we need history, branches and merges?
What does Git actually store?

Questions?